

PROJECT X

Architecture Bible v1.0

Teaching Operating System for Curriculum, Assessment, Evidence, AI and Analytics

Field	Value
Owner	Rolando Valdes Hernandez
Platform	Project X - Professional Teaching Operating System
Version	Architecture Bible v1.0
Baseline	Project X v3.0 Sprint 05
Date	2026-06-29
Purpose	Define the long-term architecture before further coding.

1. Executive Summary

Project X has evolved from a professional portfolio into a teaching operating system. The purpose of this document is to freeze the architecture before further feature development, so that every future sprint contributes to one coherent platform rather than a collection of unrelated pages.

The core design principle is separation of reusable curriculum assets from academic-year delivery. Units, lessons, objectives, resources and assessment templates are reusable professional assets. Dates taught, classes, scores, completion notes and evidence records belong to a specific academic year.

Core statement

Project X is an integrated teaching operating system where curriculum, delivery, assessment, teaching evidence, AI tools and analytics share the same underlying data model.

Immediate architectural priorities

- Treat academic years as the central organisational layer.
- Convert existing spreadsheet knowledge into a reusable curriculum library.
- Avoid duplicating data between modules; reference shared entities instead.
- Use JSON as the initial portable data layer while keeping the design ready for a future database.
- Build new features through the shared Project X shell and workspace engine, not through standalone pages.

2. Product Vision

Project X should replace a traditional CV, a lesson tracker, a curriculum folder, an assessment spreadsheet, a resource index and a teaching evidence folder with one integrated professional platform.

The product should feel closer to Microsoft Office, VS Code, Notion or Power BI than to a website. It should be modular, fast, consistent, searchable, reusable and evidence-rich.

3. Guiding Principles

Every feature must serve daily teaching, professional evidence or future reuse. If a feature does not reduce workload, improve organisation, strengthen evidence or improve decision-making, it should not be built yet.

Project X should be framework first. Pages are temporary; modules and reusable views are the long-term model.

Principle	Meaning	Design implication
Single source of truth	Each data item should live once and be referenced elsewhere.	Lessons are defined in lessons.json and linked to delivery records, assessments and evidence.
Reusable before yearly	Curriculum content must survive from one academic year to the next.	Units and lessons are stored separately from dates taught and student marks.

Framework first	The shell, navigation and workspace are shared across all modules.	New modules should use ProjectXApp and the shared workspace.
Progressive enhancement	Static JSON first, local persistence second, database later.	Avoid over-engineering before the product proves its workflows.
Professional evidence by design	Teaching evidence should be generated naturally from daily workflow.	Observations, lesson plans and standards mapping reference curriculum and delivery data.

4. System Architecture

Project X is organised into five layers: presentation shell, module views, services, data, and export/import. Each layer should have a clear responsibility.

Project X Architecture

User Interface Layer

- Shared header
- Shared sidebar
- Workspace engine
- Module navigation

Module Layer

- Curriculum Centre
- Assessment Centre
- Revision Centre
- Question Banks
- Teaching Tools
- Evidence Centre
- AI Lab
- Analytics
- Administration

Service Layer

- Registry service
- Curriculum service
- Delivery service
- Assessment service
- Evidence service
- Import service
- Export service

Data Layer

- Curriculum library
- Academic year delivery
- Assessment records
- Evidence records
- Settings and navigation

Import / Export Layer

- Excel import
- CSV import
- JSON backup
- PDF export
- Word export

Layer	Responsibility	Examples
Shell	Provides the persistent application frame.	Sidebar, header, workspace, command surface.
Modules	Provide domain-specific screens and workflows.	Curriculum dashboard, lesson library, assessment markbooks.
Services	Load, transform, validate and save data.	curriculum-service.js, import-service.js.
Data	Store reusable and year-specific records.	units.json, delivery/2025-2026.json.
Exports	Generate professional outputs.	Evidence packs, lesson plan PDFs, curriculum maps.

5. Target Folder Structure

```

projectx/
  index.html
  assets/
    projectx.css
    icons/
  core/
    app-shell.js
    bootstrap.js
    projectx-loader.js
    registry-engine.js
    router.js
    workspace.js
    state-manager.js
    event-bus.js
    theme-manager.js
  components/
    header/
    sidebar/
    cards/
    tables/
    dialogs/
    forms/
    widgets/
  apps/
    curriculum/
    assessment/
    revision/
    question-banks/

```

```

tools/
evidence/
ai-lab/
analytics/
admin/
services/
  registry-service.js
  curriculum-service.js
  delivery-service.js
  assessment-service.js
  evidence-service.js
  import-service.js
  export-service.js
  storage-service.js
config/
  navigation.json
  module-navigation.json
  settings.json
  permissions.json
data/
  curriculum/
    units.json
    lessons.json
    resources.json
    objectives.json
    vocabulary.json
    assessments.json
  delivery/
    2025-2026.json
    2026-2027.json
  assessment/
    2025-2026.json
  evidence/
    qts.json
    observations.json
    teacher-standards.json
  imports/
    import-history.json
    mappings.json

```

6. Data Model

The data model separates durable curriculum assets from annual operational records. This is the most important design decision in the platform.

Entity	Reusable?	Academic-year specific?	Notes
Subject	Yes	No	Computer Science, ICT, Digital Technology.
Key Stage	Yes	No	KS2, KS3, KS4, KS5.

Year Group	Yes	No	Year 5 to Year 13.
Unit	Yes	No	Core reusable curriculum object.
Lesson	Yes	No	Reusable lesson concept connected to a unit.
Resource	Yes	No	Slides, worksheets, code, links and teacher notes.
Assessment Template	Yes	No	Reusable assessment design or quiz.
Class Group	No	Yes	Example: 7A, 8B, Year 10 GCSE.
Delivery Record	No	Yes	Date planned, date taught, completion status, notes.
Student Score	No	Yes	Assessment mark for one student in one academic year.
Evidence Record	Usually no	Usually yes	Observation, reflection, standard mapping, lesson evidence.

6.1 Curriculum Unit

A unit is a reusable block of curriculum. It can be delivered many times across different academic years, classes and contexts.

6.2 Lesson

A lesson is a reusable teaching asset. It may include objectives, starter, teaching sequence, resources, misconceptions, homework, assessment links and evidence links.

6.3 Delivery Record

A delivery record represents a lesson being taught to a class in a specific academic year. It records dates, status, notes and evidence.

6.4 Assessment Record

Assessment records should reference units, lessons and delivery records instead of duplicating topic names manually.

6.5 Evidence Record

Evidence should be generated from real teaching workflow: lesson delivered, observation completed, reflection written, standard mapped.

Example: unit object

```
{
  "id": "unit-y7-programming-foundations",
  "title": "Programming Foundations",
  "yearGroup": "Year 7",
```

```

"keyStage": "KS3",
"subject": "Computer Science",
"sequence": 1,
"status": "Live",
"description": "Introductory programming concepts including sequence, input, variables and selection.",
"lessonIds": [
  "lesson-y7-if-statements",
  "lesson-y7-variables-inputs"
]
}

```

Example: lesson object

```

{
  "id": "lesson-y7-if-statements",
  "unitId": "unit-y7-programming-foundations",
  "title": "Selection with IF Statements",
  "yearGroup": "Year 7",
  "durationMinutes": 45,
  "learningObjective": "Use IF statements to make a program respond to different inputs.",
  "resources": ["Python editor", "Starter questions", "Worked example"],
  "status": "Ready"
}

```

Example: delivery record

```

{
  "id": "delivery-2025-2026-y7-if-statements-001",
  "academicYear": "2025-2026",
  "classGroup": "Year 7",
  "lessonId": "lesson-y7-if-statements",
  "plannedDate": "2026-06-24",
  "taughtDate": "2026-06-24",
  "status": "Completed",
  "notes": "Lesson delivered as part of QTS evidence sequence."
}

```

7. Academic Year Lifecycle

Academic years should be treated as first-class objects. A new academic year should not require recreating curriculum content. It should create new delivery, assessment and evidence records that reference the existing curriculum library.

1. Create or select academic year.
2. Choose curriculum library to use as the base.
3. Copy unit and lesson sequence by reference, not by duplication.
4. Create empty delivery plans for selected classes.
5. Create assessment templates for the year.
6. Do not copy student marks, teaching notes or completed evidence unless explicitly requested.
7. Begin delivery tracking for the new year.

Copy into new year	Recommended default	Reason
Units	Yes, by reference	Curriculum sequence is reusable.
Lessons	Yes, by reference	Lesson assets remain stable.
Resources	Yes, by reference	Avoid duplicate files.
Planned teaching dates	Optional	School timetable changes yearly.
Student names	No	Classes change.
Student marks	No	Assessment data is year-specific.
QTS evidence	No	Professional evidence belongs to the year/context in which it was created.

8. Import Centre

The Lesson Tracker workbook should become the seed import source for Project X. The import system should detect sheets, rows, units, lessons, resources, assessment rows and delivery information, then convert them into the Project X data model.

Import source	Initial support	Future support
Lesson Tracker Excel workbook	Yes	Ongoing synchronisation and change detection.
CSV export	Yes	Generic tabular import.
Google Sheets	Manual export first	Direct API connection later.
OCR/AQA scheme files	Manual mapping first	Exam-board template import later.
Project X backup	Yes	Restore and migration workflow.

8.1 Import workflow

- Upload or select a workbook.
- Detect worksheets such as Y5, Y6, Y7, Y8, Y9, Y10, Y11, Y12, NEA and Y13.
- Preview detected units, lessons, assessments and resources.
- Allow the user to map ambiguous columns.
- Generate reusable curriculum JSON.
- Generate delivery JSON for the academic year represented in the workbook.
- Generate assessment JSON where marks or assessment rows are present.
- Record the import in import-history.json.
- Allow rollback if the import was wrong.

Import output target

```
projectx/data/curriculum/units.json
projectx/data/curriculum/lessons.json
projectx/data/curriculum/resources.json
projectx/data/delivery/2025-2026.json
projectx/data/assessment/2025-2026.json
projectx/data/imports/import-history.json
```


9. Module Architecture

Module	Purpose	Initial views
Curriculum Centre	Manage reusable units, lessons, schemes, resources and yearly delivery.	Unit Manager, Lesson Library, Planner, Resources, Coverage.
Assessment Centre	Connect assessments to curriculum and students.	Markbooks, gradebook, feedback, targets, reports.
Revision Centre	Generate and organise revision pathways from curriculum and assessment gaps.	Topic checklists, retrieval practice, exam prep.
Question Banks	Store reusable questions linked to topics, difficulty and exam boards.	GCSE, A Level, retrieval, quizzes.
Teaching Evidence Centre	Turn teaching work into professional evidence.	QTS evidence, observations, teacher standards, reflections.
AI Lab	Use project data as context for AI-assisted resource generation.	Lesson generator, worksheet generator, feedback writer.
Analytics	Show operational and professional insights.	Coverage, assessment trends, evidence summaries.
Administration	Manage settings, registry, imports and backups.	Settings, navigation, import history, backups.

10. Curriculum Centre Specification

Curriculum Centre is the flagship operational module. It should be the first full module to become genuinely usable.

View	Purpose	Data used
Dashboard	KPIs and overview of units, lessons, resources and key stages.	units.json, lessons.json, resources.json.
Units	Create, edit, search and inspect units.	units.json.
Lessons	Create, edit and link lessons to units.	lessons.json, units.json.
Schemes of Work	Assemble units and lessons into teachable sequences.	units.json, lessons.json, delivery plan.
Resources	Manage reusable resource references.	resources.json.
Planner	Plan delivery by class and academic year.	delivery/YYYY-YYYY.json.
Settings	Curriculum-specific preferences.	settings.json.

11. Assessment Centre Specification

Assessment Centre should not become a separate spreadsheet clone. Its value comes from linking marks to curriculum and lesson context.

- Every assessment should reference a unit, lesson or topic.
- Every markbook should belong to an academic year and class group.
- Feedback should reference skills, misconceptions and curriculum gaps.
- Analytics should identify topic-level strengths and weaknesses.

Assessment record model

```
{
  "id": "assessment-y8-data-representation-001",
  "academicYear": "2025-2026",
  "classGroup": "Year 8",
  "unitId": "unit-y8-data-representation",
  "title": "Data Representation Assessment",
  "maxScore": 40,
  "date": "2026-05-15",
  "scores": [
    { "studentId": "student-001", "score": 32, "feedback": "Secure binary conversion." }
  ]
}
```

12. Teaching Evidence Centre Specification

Teaching Evidence Centre should become a living evidence system. The key is that evidence is connected to real curriculum and delivery records rather than being stored as isolated portfolio pages.

Evidence type	References	Use
Observation	lessonId, deliveryId, standardIds	Professional observation evidence.
Lesson plan	lessonId, resources	Teaching standards and planning evidence.
Reflection	observationId, standards	Reflective practice.
Student work	lessonId, classGroup	Progress and assessment evidence.
Mentor feedback	evidenceId, standardIds	Professional development.
QTS report	standardIds, finalStatus	Formal qualification evidence.

13. AI Lab Strategy

AI Lab should not be a generic prompt box. It should use Project X data as context. The long-term value is that Project X already knows the curriculum, class, lesson sequence, objectives, resources, assessments and evidence needs.

AI tool	Inputs from Project X	Output
---------	-----------------------	--------

Lesson Generator	Unit, lesson objective, year group, prior lesson, resources.	Lesson plan and slide outline.
Worksheet Generator	Lesson, objective, misconceptions, difficulty.	Printable worksheet and answers.
Feedback Writer	Assessment scores, rubric, topic.	Personalised feedback comments.
Report Comment Generator	Assessment, ATL notes, curriculum progress.	Report comments.
Observation Reflection Assistant	Lesson, observation notes, standards.	Professional reflection draft.

14. Analytics Strategy

Analytics should emerge from normal Project X usage. If the curriculum, delivery, assessment and evidence records are connected, analytics becomes a by-product rather than another manual task.

- Curriculum coverage by year group, key stage and topic.
- Completed vs planned lessons by class.
- Assessment performance by unit and topic.
- Evidence coverage against Teacher Standards.
- Resource readiness for upcoming lessons.
- Workload indicators: reports due, assessments pending, evidence gaps.

15. UI/UX Design Language

Design element	Standard
Colour palette	Blue and white as primary; soft blue backgrounds; green only for success/accent.
Layout	Persistent sidebar, shared header, central workspace.
Cards	Rounded cards with subtle borders and shadows.
Tables	Clean, readable, sortable, with clear headers.
Forms	Two-column where appropriate; clear labels; primary action at bottom/right.
Navigation	Primary sidebar for modules; secondary navigation for module views.
Tone	Professional, minimal, software-like.

16. Coding Standards

- Use complete replacement files for releases to avoid mixed versions.
- Keep core files generic; do not hard-code module-specific behaviour inside the core unless absolutely necessary.
- Use services for data loading and transformation.

- Use configuration files for navigation, registry and settings.
- Prefer semantic IDs and stable keys over display names.
- Keep data portable and human-readable while the system remains JSON-based.
- Every sprint should include release notes, changelog, install instructions and test checklist.

Naming standards

Files: kebab-case.js, kebab-case.json, index.html

Classes: PascalCase, e.g. ProjectXApp

Functions: camelCase, e.g. loadCurriculumData

IDs: stable semantic strings, e.g. unit-y8-data-representation

JSON keys: camelCase, e.g. yearGroup, keyStage, lessonIds

17. Roadmap

Version/Sprint	Goal	Outcome
v3 Sprint 06	Import Centre and Academic Year Engine	Convert workbook data into reusable curriculum and yearly delivery structures.
v3 Sprint 07	Curriculum CRUD	Create/edit/delete units and lessons with local persistence.
v3 Sprint 08	Planner and New Academic Year	Generate next academic year delivery plan from reusable curriculum.
v3 Sprint 09	Assessment Centre Foundation	Markbooks connected to curriculum.
v3 Sprint 10	Evidence Centre 2.0	Teacher Standards mapping linked to lesson and observation data.
v4.0	Teaching Operating System Beta	Integrated curriculum, delivery, assessment, evidence and AI workflows.
v5.0	Portable Product Architecture	Backup/restore, plugin system, possible multi-user readiness.

18. Risks and Controls

Risk	Impact	Control
Data duplication	Inconsistent curriculum and delivery records.	Single source of truth and reference IDs.
Too many features too early	Technical debt and confusing UI.	Sprint discipline and architecture freeze.
Excel import ambiguity	Incorrect unit/lesson detection.	Preview, mapping and rollback.
Browser storage limitations	Data loss or device-specific records.	JSON export/import and later database migration.
Hard-coded paths	Local/GitHub mismatch.	Central environment config and path helpers.

Portfolio vs product confusion	Inconsistent design.	Project X shell governs all modules.
--------------------------------	----------------------	--------------------------------------

19. Release Process

Each future sprint should be delivered as a complete release package, not as isolated snippets. This reduces the chance of partial upgrades and keeps the platform consistent.

```
ProjectX_v3_SprintXX_release/
projectx/
data/
RELEASE_NOTES.md
CHANGELOG.md
INSTALL.md
TEST_CHECKLIST.md
```

File	Purpose
RELEASE_NOTES.md	User-facing summary of what changed.
CHANGELOG.md	Technical record of changed files and architecture changes.
INSTALL.md	Replacement instructions.
TEST_CHECKLIST.md	Verification steps before declaring the sprint complete.

20. Architecture Decisions Locked for v1.0

17. Project X is a teaching operating system, not a website.
18. Academic years are central to the system model.
19. Reusable curriculum is separate from yearly delivery.
20. Assessment, evidence, analytics and AI should reference shared curriculum data.
21. The Project X shell and workspace engine are the foundation for all modules.
22. The Lesson Tracker workbook should be treated as a seed import source, not the long-term master database.
23. Future development should be release-based with complete replacement packages.

This document should be treated as the working blueprint for Project X from Sprint 06 onward. It can evolve, but future changes should be intentional architectural decisions rather than ad hoc page edits.

Appendix A - Detailed Data Dictionary

This appendix defines the expected fields for the first Project X JSON data layer. These structures are intentionally simple and portable, while still supporting future migration to a relational or document database.

Academic Year

Field	Description
id	Stable ID such as academic-year-2025-2026
label	Human readable label, e.g. 2025-2026
startDate	Academic year start date
endDate	Academic year end date
status	active, archived, draft
source	manual, imported, generated

Unit

Field	Description
id	Stable ID
title	Unit title
subject	Subject name
keyStage	KS2-KS5
yearGroup	Year 5-Year 13
sequence	Numerical ordering
description	Unit overview
status	draft, live, retired
lessonIds	Array of linked lesson IDs

Lesson

Field	Description
id	Stable ID
unitId	Parent unit
title	Lesson title
durationMinutes	Planned duration
learningObjective	Primary objective
successCriteria	Optional list
resources	Array of resource IDs or labels
misconceptions	Optional list
status	draft, ready, evidence

Resource

Field	Description
id	Stable ID
title	Resource title
type	slides, worksheet, code, link, video, assessment
url	Optional link

lessonIds	Lessons using this resource
tags	Search tags
status	draft, live, archived

Delivery Record

Field	Description
id	Stable ID
academicYear	Year label
classGroup	Class/group
lessonId	Referenced lesson
plannedDate	Planned teaching date
taughtDate	Actual date
status	planned, completed, moved, cancelled
notes	Teacher notes

Assessment

Field	Description
id	Stable ID
academicYear	Year label
classGroup	Class/group
unitId	Linked unit
lessonId	Optional linked lesson
title	Assessment title
maxScore	Maximum mark
scores	Array of student score records

Evidence

Field	Description
id	Stable ID
type	observation, reflection, student work, mentor feedback
lessonId	Optional linked lesson
deliveryId	Optional linked delivery
standardIds	Teacher Standards links
summary	Evidence summary
url	Optional file/link

Appendix B - Lesson Tracker Import Rules

The current Lesson Tracker workbook should be treated as a historical and operational import source. Import rules must prioritise preserving reusable curriculum content while separating delivery and assessment records.

Workbook signal	Project X interpretation	Target file
Worksheet name: Y5, Y6, Y7 etc.	Year group context for detected rows.	units.json, lessons.json, delivery year file.
Worksheet name: NEA	Coursework / project evidence area.	assessment and evidence data.
Rows containing unit names	Potential reusable curriculum units.	curriculum/units.json.
Rows containing lesson titles	Reusable lesson records.	curriculum/lessons.json.
Dates in lesson rows	Delivery records for a specific year.	delivery/2025-2026.json.
Max score / marks columns	Assessment records.	assessment/2025-2026.json.
Links, resource names, file names	Resource records or resource tags.	curriculum/resources.json.
Comments / notes	Delivery or evidence notes depending on context.	delivery or evidence JSON.

Import validation checklist

24. Confirm workbook year or infer from file name.
25. Confirm worksheet-to-year-group mapping.
26. Detect duplicated lesson titles and propose merges.
27. Flag ambiguous rows before import.
28. Preview counts before writing data.
29. Allow rollback by saving import snapshot.
30. Record import date, source file name and resulting counts.
31. Never overwrite master curriculum without a preview step.

Appendix C - Core User Workflows

Create New Academic Year

32. Open Administration > Academic Years
33. Select Create New Year
34. Choose source curriculum library
35. Choose whether to copy resources and assessment templates
36. Confirm that marks and historical notes will not be copied
37. Generate delivery skeleton
38. Review and activate year

Import Existing Lesson Tracker

39. Open Import Centre
40. Choose Lesson Tracker workbook
41. Preview detected sheets
42. Map columns if required
43. Preview units, lessons, resources and assessments
44. Import to curriculum and year delivery data
45. Review import history

Plan a Lesson

46. Open Curriculum > Planner
47. Select academic year and class
48. Choose unit and lesson
49. Set planned date
50. Attach resources
51. Save delivery plan
52. Optional: generate starter/worksheet from AI Lab

Record Lesson Completion

53. Open today or class view
54. Mark lesson completed
55. Add brief notes
56. Attach evidence if required
57. Update coverage analytics automatically

Create Evidence Record

58. Open lesson or delivery record
59. Choose Add Evidence
60. Select evidence type
61. Map Teacher Standards
62. Add reflection or file link
63. Save to Evidence Centre

Generate AI Resource

64. Open lesson
65. Choose AI action
66. Confirm objective, year group and context
67. Generate output
68. Review and save as resource

Appendix D - Screen and View Inventory

Module	View	Purpose	Build priority
Dashboard	Home	Global project overview, recent activity, quick links.	High
Curriculum	Dashboard	Curriculum KPIs and latest items.	Done / ongoing
Curriculum	Units	Manage reusable units.	High
Curriculum	Lessons	Manage reusable lessons.	High
Curriculum	Planner	Plan delivery by class and academic year.	High
Curriculum	Resources	Manage reusable resources.	Medium
Curriculum	Schemes	Build schemes of work from units and lessons.	Medium
Import Centre	Workbook Import	Import Excel/CSV data.	High
Administration	Academic Years	Create and manage years.	High
Assessment	Markbooks	Manage marks linked to curriculum.	Medium
Evidence	Standards Map	Map evidence to Teacher Standards.	Medium
AI Lab	Lesson Generator	Generate lesson assets from curriculum context.	Medium
Analytics	Coverage	Show delivery and curriculum coverage.	Medium

Appendix E - Technical Implementation Plan

The following implementation sequence should guide Sprint 06 onward. It prevents the project from building isolated screens before the data model and import mechanics are ready.

Sprint	Primary technical work	User-facing result
Sprint 06	Import service, academic-year model, import history.	Import centre detects Lesson Tracker and creates clean JSON.
Sprint 07	Storage service, CRUD helpers, validation utilities.	Units and lessons can be created, edited and deleted safely.
Sprint 08	Planner service and delivery views.	Academic year delivery plan is generated from reusable curriculum.
Sprint 09	Assessment service and markbook model.	Assessments link to units/lessons and calculate basic summaries.
Sprint 10	Evidence service and standards mapping.	Teaching evidence attaches to lessons and standards.
Sprint 11	AI context builder.	AI tools use lesson/unit data automatically.
Sprint 12	Analytics aggregation service.	Coverage and progress analytics become automatic.

Appendix F - Service Contracts

curriculum-service.js

- loadUnits()
- loadLessons()
- getUnitById(id)
- getLessonsByUnit(unitId)
- saveUnit(unit)
- saveLesson(lesson)
- deleteUnit(id)
- deleteLesson(id)

delivery-service.js

- loadAcademicYear(year)
- createAcademicYear(year, options)
- planLesson(deliveryRecord)
- markCompleted(deliveryId, notes)
- getDeliveryByClass(classGroup)

import-service.js

- detectWorkbook(file)
- previewImport(file)
- mapColumns(mapping)
- runImport(options)
- rollbackImport(importId)

assessment-service.js

- loadAssessments(year)
- createAssessment(assessment)
- recordScore(assessmentId, studentScore)
- calculateSummary(assessmentId)

evidence-service.js

- loadEvidence()
- createEvidence(record)
- mapStandards(evidenceId, standardIds)
- getEvidenceByLesson(lessonId)

analytics-service.js

- calculateCoverage(year)
- calculateAssessmentSummary(year)
- calculateEvidenceCoverage(year)
- buildDashboardKpis()

storage-service.js

- readJson(path)
- writeJson(path, data)
- readLocal(key)
- writeLocal(key, data)
- exportBackup()
- importBackup(file)

Appendix G - JSON File Schemas

units.json

```
[
  {
    "id": "unit-y7-programming-foundations",
    "title": "Programming Foundations",
    "subject": "Computer Science",
    "keyStage": "KS3",
    "yearGroup": "Year 7",
    "sequence": 1,
    "description": "...",
    "status": "Live",
    "lessonIds": []
  }
]
```

lessons.json

```
[
  {
    "id": "lesson-y7-if-statements",
    "unitId": "unit-y7-programming-foundations",
    "title": "Selection with IF Statements",
    "durationMinutes": 45,
    "learningObjective": "...",
    "resources": [],
    "status": "Ready"
  }
]
```

delivery/2025-2026.json

```
[
  {
    "id": "delivery-001",
    "academicYear": "2025-2026",
    "classGroup": "Year 7",
    "lessonId": "lesson-y7-if-statements",
    "plannedDate": "2026-06-24",
    "taughtDate": "2026-06-24",
    "status": "Completed",
    "notes": "..."
  }
]
```

import-history.json

```
[
  {
    "id": "import-2026-06-29-001",
    "sourceFile": "Lesson Tracker 2025-2026.xlsx",
    "importedAt": "2026-06-29T10:00:00Z",
    "status": "Success",
    "counts": {
```

```
"units": 0,  
"lessons": 0,  
"resources": 0,  
"assessments": 0  
}  
}  
]
```


Appendix H - Testing Strategy

Test area	Checks
Navigation	Sidebar active states, dashboard links, module routes, browser back/forward.
Data loading	JSON files load correctly on GitHub Pages and local server.
Import	Preview counts match workbook expectations; rollback works.
CRUD	Create/edit/delete operations update UI and local persistence.
Responsive layout	Dashboard and modules work on desktop, tablet and phone widths.
Evidence	Evidence records keep links to lesson, delivery and standards.
Accessibility	Keyboard access, labels, contrast, semantic headings.
Regression	Old portfolio pages still open after Project X updates.

Definition of done for future sprints

- All changed files are delivered in a release ZIP.
- CHANGELOG, INSTALL and TEST_CHECKLIST are included.
- Core shell still loads on GitHub Pages.
- Dashboard and at least one affected module are manually verified.
- No duplicate or obsolete scripts are knowingly introduced.
- Data changes are backward-compatible or migration steps are documented.

Appendix I - Future Database Migration

Project X can begin with static JSON because GitHub Pages is simple and portable. However, the architecture should stay ready for a future database when persistence, authentication or multi-device editing become necessary.

Phase	Storage model	When appropriate
Phase 1	Static JSON + browser localStorage	Current GitHub Pages prototype.
Phase 2	JSON backup/export/import	When real data needs portability and safety.
Phase 3	IndexedDB in browser	When offline-first editing becomes important.
Phase 4	Hosted database/API	When multi-device sync or multi-user access is needed.
Phase 5	Full SaaS architecture	If Project X becomes a product for other teachers.

Current JSON entity	Future database table/collection
units.json	units
lessons.json	lessons
resources.json	resources
delivery/YYYY.json	delivery_records
assessment/YYYY.json	assessments and scores
evidence/*.json	evidence_records
module-navigation.json	navigation_config

Appendix J - Accessibility and Professional Standards

- Use real headings in logical order.
- All form controls must have visible labels.
- All buttons must have clear action text.
- Tables should have header rows and not rely only on colour.
- Keyboard users should be able to navigate module menus and forms.
- Status labels should use text as well as colour.
- Contrast should remain strong against the blue/white theme.
- Exports should be suitable for professional review or evidence submission.

Appendix K - Glossary

Term	Definition
Academic Year	A school year such as 2025-2026. It owns delivery, marks and year-specific evidence.
Curriculum Library	The reusable master set of units, lessons, resources and objectives.
Delivery Record	A record that a reusable lesson was planned or taught to a class in a particular year.
Evidence Record	Professional evidence linked to lessons, observations or teacher standards.
Module	A major Project X application area such as Curriculum, Assessment or AI Lab.
View	A screen within a module, such as Units or Lessons.
Workspace	The central area where module views are rendered.
Registry	The metadata list describing Project X applications and links.
Import Centre	The module responsible for converting external files into Project X data.
Single Source of Truth	A design rule stating that each piece of information should be stored once and referenced elsewhere.

Appendix L - Sprint 06 Starting Brief

The next coding sprint should implement the Import Centre and Academic Year Engine. This sprint should be treated as the first sprint based on the Architecture Bible.

Deliverable	Description
Import Centre view	A module view for selecting and previewing Lesson Tracker imports.
Academic Year Manager	Create/select academic years and initialise delivery records.
Import history	Record source file, date, status and import counts.
Mapping preview	Show detected sheets and proposed unit/lesson counts.
Curriculum data update	Write or simulate updates to curriculum JSON structures.
Test data	Use a representative extract of the Lesson Tracker workbook.

This Architecture Bible should be updated after major architecture decisions, but not after every small sprint. It is the product blueprint and should remain stable enough to guide future coding.